Abaixo você tem **todo o MVP completo**, organizado exatamente como você usaria no seu padrão AWS + IaC. Basta copiar e colar em arquivos locais.

📦 Estrutura do Projeto (MVP Agras)

```
agrotech-dji-agras-mvp/
│
├── infra/
│   └── cloudformation.yaml
│
├── lambda/
│   └── processor.py
│
├── data/
│   └── sample_flight.json
│
├── athena/
│   ├── create_table.sql
│   └── views.sql
│
├── README.md
└── DEPLOY.md
```

☁️ 1) CloudFormation (Data Lake + Pipeline)

```yaml
AWSTemplateFormatVersion: '2010-09-09'
Description: MVP Agrotech DJI Agras Telemetry

Parameters:
  Project:
    Type: String
    Default: agrotech
  Environment:
    Type: String
    Default: dev

Resources:

  RawBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: !Sub "${Project}-${Environment}-agras-raw"

  ProcessedBucket:
    Type: AWS::S3::Bucket
    Properties:
```

```
    BucketName: !Sub "${Project}-${Environment}-agras-processed"

  LambdaRole:
    Type: AWS::IAM::Role
    Properties:
      RoleName: !Sub "${Project}-${Environment}-lambda-role"
      AssumeRolePolicyDocument:
        Statement:
          - Effect: Allow
            Principal:
              Service: lambda.amazonaws.com
            Action: sts:AssumeRole
      Policies:
        - PolicyName: s3-access
          PolicyDocument:
            Statement:
              - Effect: Allow
                Action: s3:*
                Resource: "*"

  ProcessorLambda:
    Type: AWS::Lambda::Function
    Properties:
      FunctionName: !Sub "${Project}-${Environment}-processor"
      Runtime: python3.13
      Handler: processor.lambda_handler
      Role: !GetAtt LambdaRole.Arn
      Timeout: 60
      Code:
        ZipFile: |
          # placeholder - deploy real code manually

Outputs:
  RawBucket:
    Value: !Ref RawBucket
  ProcessedBucket:
    Value: !Ref ProcessedBucket
```

🧠 2) Lambda Processor (normalização + score)

```
import json
import boto3
from datetime import datetime

s3 = boto3.client("s3")

def calculate_score(record):
```

```
    score = 100

    if record["speed_mps"] > 7:
        score -= 10

    if record["battery"] < 20:
        score -= 20

    if record["altitude"] < 2:
        score -= 5

    return max(score, 0)

def lambda_handler(event, context):

    for record in event["Records"]:
        bucket = record["s3"]["bucket"]["name"]
        key = record["s3"]["object"]["key"]

        obj = s3.get_object(Bucket=bucket, Key=key)
        data = json.loads(obj["Body"].read())

        processed = []

        for r in data:
            processed.append({
                "flight_id": r["flight_id"],
                "timestamp": r["timestamp"],
                "lat": r["lat"],
                "lon": r["lon"],
                "speed_mps": r["speed_mps"],
                "battery": r["battery"],
                "score": calculate_score(r)
            })

        output_key = key.replace("raw", "processed")

        s3.put_object(
            Bucket=bucket.replace("raw", "processed"),
            Key=output_key,
            Body=json.dumps(processed)
        )

    return {"status": "ok"}
```

📊 3) Dataset Sintético (simulação Agras)

```
[
 {
  "flight_id": "AGRAST40_001",
  "timestamp": "2026-04-20T14:00:00Z",
  "lat": -19.92,
  "lon": -44.05,
  "altitude": 3.2,
  "speed_mps": 5.5,
  "battery": 85
 },
 {
  "flight_id": "AGRAST40_001",
  "timestamp": "2026-04-20T14:00:01Z",
  "lat": -19.921,
  "lon": -44.051,
  "altitude": 3.1,
  "speed_mps": 6.1,
  "battery": 84
 }
]
```

🗄️ 4) Athena – Tabela

```
CREATE EXTERNAL TABLE agras_telemetry (
 flight_id STRING,
 timestamp STRING,
 lat DOUBLE,
 lon DOUBLE,
 speed_mps DOUBLE,
 battery INT,
 score INT
)
ROW FORMAT SERDE 'org.openx.data.jsonserde.JsonSerDe'
LOCATION 's3://agrotech-dev-agras-processed/';
```

📈 5) Views Analíticas

```
CREATE VIEW vw_flight_summary AS
SELECT
 flight_id,
 COUNT(*) AS total_points,
 AVG(speed_mps) AS avg_speed,
 MIN(battery) AS min_battery,
 AVG(score) AS avg_score
FROM agras_telemetry
GROUP BY flight_id;
```

🚀 6) DEPLOY (sequência prática)

```
# 1. Criar stack
aws cloudformation deploy \
  --template-file infra/cloudformation.yaml \
  --stack-name agrotech-agras-mvp \
  --capabilities CAPABILITY_NAMED_IAM

# 2. Upload dataset
aws s3 cp data/sample_flight.json \
  s3://agrotech-dev-agras-raw/raw/sample.json

# 3. Criar tabela no Athena
# (rodar SQL manual ou via CLI)

# 4. Visualizar no QuickSight
```

📊 7) Dashboard sugerido

Com Amazon QuickSight:

- KPI: área estimada / tempo
- Score médio por voo
- Mapa (lat/lon)
- Consumo vs eficiência
- Comparação entre missões

🤖 8) Evolução com IA (próximo passo)

Com Amazon Web Services Bedrock:

- “Explique por que essa missão teve score baixo”
- “Sugira melhorias”
- “Compare operadores”

🎯 9) Valor estratégico (o que você pode vender)

Você já consegue demonstrar:

- rastreabilidade completa da missão
- eficiência operacional

- redução de desperdício
- análise comparativa entre voos
- base para agricultura de precisão